

# Table of contents

|                                                      |          |
|------------------------------------------------------|----------|
| <b>TASK 1: THE LONG-RUNNING PROCESS</b>              | <b>3</b> |
| A. C Source Code                                     | 3        |
| B. Terminal Execution & Verification                 | 3        |
| <b>TASK 2: PROCESS IDENTITY (PID AND PPID)</b>       | <b>4</b> |
| A. C Source Code                                     | 4        |
| B. Terminal Execution & Verification                 | 4        |
| <b>TASK 3: EXIT CODES AND OS FEEDBACK</b>            | <b>5</b> |
| A. C Source Code                                     | 5        |
| B. Terminal Execution & Verification                 | 6        |
| <b>TASK 4: STANDARD I/O STREAMS</b>                  | <b>6</b> |
| A. C Source Code                                     | 6        |
| B. Terminal Execution                                | 7        |
| <b>TASK 5: CONDITIONAL EXECUTION AND TERMINATION</b> | <b>7</b> |
| A. C Source Code                                     | 7        |
| B. Terminal Execution & Verification                 | 8        |
| <b>CONCLUSION</b>                                    | <b>8</b> |

# Table of figures

|                                                                |   |
|----------------------------------------------------------------|---|
| Figure 1: task1_alive.c.....                                   | 3 |
| Figure 2: Run in background.....                               | 3 |
| Figure 3: monitor the process.....                             | 4 |
| Figure 4 : task2_identity.c.....                               | 4 |
| Figure 5: Terminal command for execution and verification..... | 5 |
| Figure 6 : task3_exit.c.....                                   | 5 |
| Figure 7: Checking Exit Status.....                            | 6 |
| Figure 8: task4_input.c.....                                   | 6 |
| Figure 9 : Running task4.....                                  | 7 |
| Figure 10: task5 control.c.....                                | 7 |
| Figure 11: Testing Control Flow.....                           | 8 |

## Lab 3: Investigating Process Lifecycles and OS Interaction

### TASK 1: THE LONG-RUNNING PROCESS

#### A. C Source Code

Description: Create a program that runs for 30 seconds using a loop and sleep().



```
GNU nano 8.7.1 task1_alive.c
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("I am starting...\n");

    // Loop for 30 seconds
    for (int i = 1; i <= 30; i++) {
        sleep(1);
    }

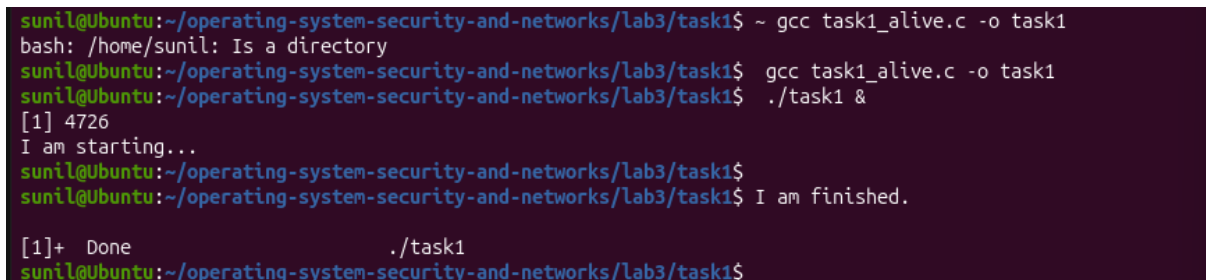
    printf("I am finished.\n");

    return 0;
}
```

Figure 1: task1\_alive.c

#### B. Terminal Execution & Verification

Description: The process is executed in the background using & and monitored using the ps command.



```
sunil@ubuntu:~/operating-system-security-and-networks/lab3/task1$ gcc task1_alive.c -o task1
bash: /home/sunil: Is a directory
sunil@ubuntu:~/operating-system-security-and-networks/lab3/task1$ gcc task1_alive.c -o task1
sunil@ubuntu:~/operating-system-security-and-networks/lab3/task1$ ./task1 &
[1] 4726
I am starting...
sunil@ubuntu:~/operating-system-security-and-networks/lab3/task1$
sunil@ubuntu:~/operating-system-security-and-networks/lab3/task1$ I am finished.

[1]+  Done                  ./task1
sunil@ubuntu:~/operating-system-security-and-networks/lab3/task1$
```

Figure 2: Run in background

```
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task1$ ps aux | grep task1
sunil      4748  0.0  0.0   9316  2672 pts/1    S+   08:10   0:00 grep --color=auto task1
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task1$
```

Figure 3: monitor the process

## TASK 2: PROCESS IDENTITY (PID AND PPID)

### A. C Source Code

Description: create the C program that sleeps for 20 seconds so that the process can be verified.

```
#include <stdio.h>
#include <unistd.h>

int main() {
    // Get current Process ID
    pid_t my_pid = getpid();

    // Get Parent Process ID
    pid_t my_ppid = getppid();

    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is: %d\n", my_ppid);

    printf("Sleeping for 20 seconds...\n");
    sleep(20);

    return 0;
}
```

Figure 4 : task2\_identity.c

### B. Terminal Execution & Verification

Description: It displays the PID and PPID to verify process identity, using ps command after compilation.

```
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task2$ nano task2_identity.c
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task2$ ls
task2_identity.c
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task2$ gcc task2_identity.c -o task2
```

```

sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task2$ ./task2 &
[1] 8986
My PID is: 8986
My Parent PID is: 8490
Sleeping for 20 seconds...
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task2$ ps -p 8986 -o pid,ppid,cmd
  PID    PPID  CMD
  8986    8490  ./task2
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task2$

```

*Figure 5: Terminal command for execution and verification*

## TASK 3: EXIT CODES AND OS FEEDBACK

### A. C Source Code

Description: Write the C program that shows success or failure on the basis of a given input number.

```

GNU nano 8.7.1                                task3_exit.c
#include <stdio.h>

int main() {
    int num;

    printf("Enter a number (positive for success, negative for fail): ");
    scanf("%d", &num);

    if (num > 0) {
        printf("Success!\n");
        return 0;    // Success
    } else {
        printf("Failure!\n");
        return 1;    // Failure/Error
    }
}

```

*Figure 6 : task3\_exit.c*

## B. Terminal Execution & Verification

Description: After the execution , the program takes the input from the user and returns a different exit code. Positive number return success and negative number return failure. And the shell exit status can be checked using 'echo \$?'.

```
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task3$ nano task3_exit.c
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task3$ gcc task3_exit.c -o task3
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task3$ ./task3
Enter a number (positive for success, negative for fail): 7
Success!
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task3$ echo $?
0
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task3$ ./task3
Enter a number (positive for success, negative for fail): -5
Failure!
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task3$ echo $?
1
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task3$
```

Figure 7: Checking Exit Status

## TASK 4: STANDARD I/O STREAMS.

### A. C Source Code

Description: C program that demonstrates standard input and output by taking the user's name with scanf() and displaying it using printf().

```
GNU nano 8.7.1 task4_input.c *
#include <stdio.h>

int main() {
    char name[50];

    printf("Enter your name: ");
    scanf("%49s", name);

    printf("Hello, %s! Welcome to OS Class.\n", name);

    return 0;
}
```

Figure 8: task4\_input.c

## B. Terminal Execution

Description: After execution, the program asks for a name and displays a greeting .

```
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task4$ touch task4_input.c
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task4$ ls
task4_input.c
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task4$ nano task4_input.c
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task4$ gcc task4_input.c -o task4
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task4$ ./task4
Enter your name: sunil
Hello, sunil! Welcome to OS Class.
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task4$
```

Figure 9 : Running task4

## TASK 5: CONDITIONAL EXECUTION AND TERMINATION

### A. C Source Code

Description: Writing C program that asked for user's permission whether to continue or exit.

```
GNU nano 8.7.1 task5_control.c *
#include <stdio.h>
#include <unistd.h>

int main() {
    int choice;

    printf("Current PID: %d\n", getpid());

    printf("Do you want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Continuing...\n");
        return 0; // Success
    } else {
        printf("Exiting...\n");
        return 1; // Failure/Abort
    }
}
```

Figure 10: task5 control.c

## B. Terminal Execution & Verification

Description: Uses user input to decide whether the process continues or exits, while displaying its PID and returning an appropriate exit code.

```
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task5$ nano task5_control.c
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task5$ gcc task5_control.c -o task5
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task5$ ./task5
Current PID: 9584
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing...
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task5$ echo $?
0
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task5$ ./task5
Current PID: 9593
Do you want to continue? (1 for Yes, 0 for No): 0
Exiting...
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task5$ echo $?
1
sunil@Ubuntu:~/operating-system-security-and-networks/lab3/task5$
```

*Figure 11: Testing Control Flow*

## CONCLUSION

This lab showed how C programs run as processes managed by the Linux operating system. By using functions such as `getpid()`, `sleep()`, and return codes, we learned how C programs work with the operating system and how Linux manages processes.